Multi-Core Partition-Based Scheduling: Theoretical Foundations and Run-Time Overheads

1st Ikramul Hassan Sazid
Electronics Engineering
Hamm-Lippstadt University of Applied Sciences
Lippstadt, Germany
ikramul-hassan.sazid@stud.hshl.de

***Abstract*—With the development of today's embedded systems with advanced computation requirements, the use of multi-core processors in hard real-time systems is becoming indispensable. While parallel processing enables managing large numbers of tasks, however, it comes with several significant scheduling problems that can lead to total system malfunctioning due to a missed deadline. In this research, the move towards multi-core technology is discussed through a comparison of the global and partitioned scheduling approaches [2]. The Partitioned Earliest Deadline First (P-EDF) approach is considered in order to study its mathematical scheduling limits [1], the First Fit Decreasing (FFD) method of allocating tasks and overhead costs involved in task migration in global scheduling [5]. Using practical RTOS overheads and considering an application example for ARINC 653 avionics system [3], the reason for the strict preference of P-EDF is demonstrated.**

***Index Terms*—Multi-core scheduling, P-EDF, G-EDF, Real-Time Operating Systems, Task partitioning, ARINC 653.**

I. Introduction

In today's embedded and automotive domain, multicore processing has become a necessity for handling huge computational loads. In safety critical and Hard Real-Time (HRT) systems where the completion of tasks depends on their strict adherence to deadlines, it is a must. There are two main types of multicore scheduling schemes namely global and partitioned scheduling [2].

There are a few most common "partitioned scheduling" is practiced in the field by computer scientists and engineers. Some of them are hybrid between global and strict task partitioning.

Partitioned scheduling involves static assignment of tasks to particular processors and strictly local core execution. It prevents migration of tasks from one core to another core. Global EDF scheme allows task migration but P-EDF scheme is used more commonly than Global-EDF for HRT systems due to the high run-time overhead involved in global ready-queue and task migration delays [5]. However, P-EDF itself is mathematically a difficult problem because assigning tasks to several processors is akin to the well-known NP-hard problem of bin-packing [1]. Several empirical studies have shown that considering realistic OS overhead, P-EDF tends to outperform global schedulers [5].

II. Global vs Partitioned Real-Time Scheduling

In formal models of real-time scheduling, periodic or sporadic tasks have certain temporal constraints such as relative deadline D_i, worst-case execution time C_i, and period T_i. With respect to running those tasks on an m-core system, two major scheduling policies are considered: Global Scheduling (G-EDF) and Partitioned Scheduling (P-EDF) [2]. In Global Scheduling, there is only one global ready queue where tasks are free to run on any core. It allows free migration policy which means that a task can be preempted from running on one core and then scheduled to execute on another one. Though global scheduling theoretically leads to maximum usage of cores and reduces bin-packing problems, it has high costs associated with the implementation process since the tasks have to be synchronized and the caches should be managed properly [5]. In contrast to this, Partitioned Scheduling provides a separate local queue for each core along with a separate independent scheduler per each core.

III. Formal Description of the Algorithm

We find four main types of partitioned scheduling:

- **Strict Partitioned-EDF (P-EDF):** Here, tasks are static and permanently allocated to individual processors before execution takes place, using some of the bin packing heuristics like first fit decreasing (FFD) and best fit decreasing (BFD) [1]. After allocation, tasks cannot migrate across processors, while individual processors manage their own local ready queue according to the EDF algorithm.
- **Clustered Scheduling (C-EDF):** This scheduling scheme involves partitioning the multicore architecture into separate "clusters" of cores, where these clusters are usually formed based on memory levels such as L2 or L3 caches. Tasks are statically allocated to a particular cluster instead of being statically allocated to individual cores, while they are scheduled across that cluster by the EDF policy [2].
- **Semi-partitioned or Task-splitting Scheduling:** This method tries to address the problems related to wasted capacity in the partitioning technique through static allocation of most tasks, where only a very few tasks ($m-1$ tasks at maximum in case of m cores) could be split

between two processors [4]. For instance, in the $C = D$ approach, the first segment of the split task is processed by one core with modified deadline exactly equal to the computation time of that task, thus permitting migration and completion of that task on another core [4].

- **ARINC 653 Hierarchical Partition Scheduling:** A popular method used in the field of safety-critical avionics, which provides strict isolation of timing among different applications through a hierarchical schedule having two layers [3]. In the global layer of scheduling, Time Division Multiplexing (TDM) technique cyclically assigns the time slices to different partitions using various cores, whereas in the local layer, Fixed-Priority (FP) schedule independently schedules different real-time tasks within a partition's assigned time slice [3].

In the formal modeling of real-time multi-core systems, the processing workload is generally represented as a set of sporadic or periodic tasks $\tau = \{\tau_1, \tau_2, ..., \tau_n\}$. Each task τ_i is characterized by a worst-case execution time (WCET) C_i, a relative deadline D_i, and a minimum inter-arrival time or period T_i. The utilization of each task is defined as $u_i = C_i/T_i$, representing the fraction of a single processor's capacity required to execute the task. In implicit-deadline systems, where $D_i = T_i$, ensuring that the total utilization of tasks assigned to a specific processor does not exceed 1.0 (or 100

A. Phase 1: The Offline Task Partitioning Phase

The initial phase of P-EDF requires the operating system to map the set of n tasks onto the available set of m processors. Strict partitioning prohibits any task from migrating across processor boundaries once the system is actively running. Mathematically, the static assignment of tasks to different processors without overfilling beyond 100% of any single core's processing capacity is equivalent to solving a bin-packing problem. In this analogy, the processors are the "bins" of capacity 1.0, and the tasks are the "items" with size u_i.

Because the classical bin-packing problem is NP-hard in the strong sense, finding an optimal mapping solution in polynomial time is impossible unless P = NP [1]. Consequently, real-time operating systems must rely on polynomial-time approximation algorithms, known as heuristics, to optimize the load. Theoretical scheduling research has demonstrated that "Reasonable Allocation Decreasing" (RAD) heuristics are the most effective for this purpose [1]. A heuristic is classified as "reasonable" if it never fails to allocate a task when there is sufficient remaining capacity available on any of the active processors. The "decreasing" aspect mandates that tasks are sorted in non-increasing order of their utilizations (i.e., $u_1 \geq u_2 \geq \cdots \geq u_n$) prior to allocation.

The most common optimal RAD partitioning techniques practiced in the field include:

- **First-Fit Decreasing (FFD):** The task τ_i is scheduled on the very first processor P_j (where j is the lowest possible index) that has enough remaining unused capacity to accommodate u_i. FFD prioritizes filling the earliest processors to their maximum capacity before moving to the next.
- **Best-Fit Decreasing (BFD):** The algorithm evaluates all processors and schedules the task on the processor with the smallest remaining capacity that is still large enough to fit u_i. This leaves minimal fragmented space on heavily loaded processors.
- **Worst-Fit Decreasing (WFD):** The algorithm evaluates all processors and places the task on the processor with the largest remaining capacity. Unlike FFD and BFD, WFD aims to perfectly balance the load across all m cores, though it risks creating uniform fragmentation across the entire system.

Beneficial Computational Performance and Speedup Bounds: Evaluating the effectiveness of these heuristics often relies on a metric known as the "speedup factor." The speedup factor determines how much faster a processor must be for a specific heuristic to successfully schedule a task set that an optimal (but computationally impossible) algorithm could schedule on a standard processor. It has been formally proven that RAD heuristics such as FFD, BFD, and WFD feature a speedup bound of exactly $\frac{4}{3} - \frac{1}{3m}$ [1]. This mathematical boundary strictly proves their immense advantage over non-sorted allocation heuristics (such as pure First-Fit or Next-Fit without prior sorting), which suffer from significantly worse speedup bounds approaching a factor of 2 [1]. Sorting the tasks limits the risk that small tasks will scatter across multiple cores, leaving no single contiguous block of capacity large enough to accommodate a massive task processed later in the sequence.

Solving the Fragmentation Problem via FFD: Diving deeper into the First-Fit Decreasing (FFD) algorithm, this heuristic is particularly well-suited for use in P-EDF multi-core architectures. The primary danger in bin-packing is the creation of "fragmented spaces"—small, unusable percentages of CPU capacity scattered across multiple cores. By strictly enforcing a non-decreasing ordering phase, FFD focuses on packing the heaviest, most demanding tasks first. Smaller tasks are then seamlessly utilized to "fill in the gaps" left on the cores. This highly organized packing methodology prevents catastrophic fragmentation and allows system designers to mathematically guarantee schedulability up to a certain utilization ceiling. Specifically, it allows for setting a safe lower utilization bound of $\frac{m+1}{2}$ for the multicore processor [1]. As long as the total utilization of the task set remains below this threshold, FFD guarantees that all deadlines will be met without exceeding the capacity of the m cores.

B. Phase 2: Online Task Execution and Local Scheduling

Following the offline allocation of tasks, the system transitions into the execution phase. Because tasks are permanently bound to specific cores, the multi-core scheduling problem is effectively reduced to m independent single-core scheduling problems. Each processor separately implements its own preemptive Earliest-Deadline-First (EDF) scheduler.

During execution, each core maintains a local ready queue. When a task's job arrives, it is placed in its respective core's ready queue, ordered strictly by its absolute deadline (d_i). The scheduler on each core simply executes the active job with the closest absolute deadline, preempting currently running tasks if a newly arrived job has an earlier deadline.

Run-Time Overhead Advantages of Processor Local EDF: The true superiority of the partitioned approach reveals itself in the physical run-time environment of a Real-Time Operating System (RTOS). In a globally scheduled system (G-EDF), a single, centralized ready queue dictates task execution across all processors. This necessitates highly complex global lock mechanisms to prevent race conditions when multiple cores attempt to fetch the next task simultaneously [5]. Furthermore, G-EDF allows tasks to migrate from one core to another upon preemption.

Because P-EDF completely isolates task execution to a single core, it inherently neutralizes these overheads. Processor Local EDF removes Inter-Processor Interrupt (IPI) delays, which are heavily utilized in global scheduling to force remote cores to preempt their current tasks [5]. Additionally, strict partitioning ensures "cache affinity." In modern multi-core processors, fetching instructions and data from main memory is significantly slower than reading from the local L1 or L2 cache. By forcing a task to execute on the same core every time it is invoked, P-EDF allows the local cache to remain "warm" with the task's data. This essentially eliminates the Cache-Related Preemption and Migration Delay (CPMD) penalty that plods global scheduling schemes when a task is moved to a new core and is forced to rebuild its cache from scratch [2], [5].

C. Overcoming Bin-Packing Limitations: Semi-Partitioning and Task Splitting

Despite the mathematical efficiency of FFD and other RAD heuristics, strict P-EDF suffers from a critical vulnerability known as the "heavy task" problem. If a system contains tasks with individual utilizations greater than 50% ($u_i > 0.5$), bin-packing algorithms can fail to schedule the system even if the total utilization of all tasks is far below the multi-core platform's maximum capacity. For example, two tasks each requiring 60% of a core cannot be placed on the same processor, inevitably leaving 40% of two separate processors entirely wasted.

To overcome the capacity waste inherent in strict bin-packing without incurring the massive run-time overheads of global scheduling, semi-partitioning techniques such as task splitting have been developed. In a semi-partitioned scheme, the vast majority of tasks are statically assigned to individual cores exactly as they are in strict P-EDF. However, a highly restricted subset of tasks—specifically, no more than $m - 1$ tasks in an m-core system—are permitted to be split across two processors [4].

A highly effective implementation of this concept is the $C = D$ **task splitting scheme** [4]. When a heavy task τ_i cannot fit onto a single processor, it is split into two separate segments: τ_{i1} and τ_{i2}. The first segment, τ_{i1}, is allocated to one core and is granted a modified, artificial deadline that is exactly equal to its computation time ($d_{i1} = e_{i1}$). By setting the deadline equal to the computation time ($C = D$), the RTOS enforces that the first segment executes non-preemptively, effectively granting it the highest priority on that core until it completes. Once τ_{i1} finishes execution, the second segment, τ_{i2}, is released on a different core. Because τ_{i1} has finished, τ_{i2} can safely migrate and execute on the second core without any risk of concurrent execution (parallelism of the same task), which is strictly forbidden in real-time task models [4]. The distinct advantage of the $C = D$ scheme is that it mathematically guarantees optimal utilization of split tasks while requiring absolutely no specialized run-time migration mechanisms, synchronization locks, or complex OS modifications [4].

D. Alternative Partitioning Topologies: Clustered and Hierarchical Methods

While strict P-EDF and semi-partitioned EDF represent the primary focus of task-to-core mapping, it is important to contextualize them within broader formal models of partitioned scheduling:

- **Clustered Scheduling (C-EDF):** As an intermediary step between pure global and pure partitioned scheduling, C-EDF involves partitioning the multicore architecture into discrete "clusters" of cores. These clusters are practically defined by physical hardware boundaries, usually grouping cores that share an L2 or L3 memory cache [2]. Rather than binding a task to a single core (like P-EDF) or allowing it to float across all m cores (like G-EDF), tasks are statically allocated to a specific cluster. Within that cluster, tasks are scheduled using global EDF policies [2]. This hybrid approach mitigates the bin-packing fragmentation issues of P-EDF while actively constraining the cache-invalidation penalties of G-EDF to a localized, shared-memory domain.
- **ARINC 653 Hierarchical Partition Scheduling:** In the highly regulated domain of safety-critical avionics, partitioning extends beyond physical core allocation into strict temporal isolation. The ARINC 653 standard utilizes a two-layer hierarchical schedule [3]. At the global layer, the multi-core system relies on Time Division Multiplexing (TDM) to cyclically assign rigid time slices (partition windows) to distinct application partitions across various cores. At the local layer, an independent Fixed-Priority (FP) or EDF scheduler orchestrates the specific real-time tasks within that partition's designated time slice [3]. This model ensures that a fault, infinite loop, or deadline miss in one software partition cannot dynamically steal CPU cycles from another, ensuring the absolute determinism required for flight-control systems.

IV. The Schedulability Test and Real RTOS Overheads

In classical real-time scheduling theory, mathematical bounds are often derived under the assumption of a sim-

plified, ideal system architecture. These foundational models typically assume that operating system operations—such as task preemption, queue management, and context switching—consume zero execution time. However, this theoretical assumption does not reflect the physical and architectural realities of a Real-Time Operating System (RTOS) running on modern multi-core hardware [5]. Every action taken by the RTOS kernel requires CPU cycles, thereby stealing processing time away from the actual application tasks. This physical reality inflates the Worst-Case Execution Time (WCET) of every task in the system, which can catastrophically collapse the theoretical schedulability utilization ceiling [5].

To bridge the gap between abstract theory and physical implementation, system designers must integrate real RTOS overheads into formal schedulability tests. In a strictly partitioned system (P-EDF), the multi-core scheduling problem is reduced to independent single-core EDF scheduling problems. For a processor core to be deemed formally schedulable under EDF, the sum of its nominal task utilization (U_{nom}) plus its overhead utilization (U_{OH}) must safely fit within the absolute maximum capacity of that single core (which is 1.0, or 100%). When evaluating the entire m-core system during the bin-packing allocation phase, the comprehensive schedulability test is mathematically defined as:

$$U_{nom} + U_{OH} \leq m \tag{1}$$

The nominal utilization (U_{nom}) is simply the baseline computation requirement of the task set, calculated as $\sum(C_i/T_i)$, where C_i is the pure worst-case execution time and T_i is the period. However, the overhead utilization (U_{OH}) is far more complex. Summed over all tasks assigned to the system, it is mathematically modeled as the total worst-case OS delay divided by the task's period [5]:

$$U_{OH} = \sum_{i=1}^{n} \frac{\Delta_i}{T_i} \tag{2}$$

This equation highlights a critical vulnerability in real-time systems: tasks with very short periods (high-frequency tasks) suffer disproportionately from OS overheads, as the delay Δ_i is inflicted upon the system much more frequently. The term Δ_i represents the aggregate per-job worst-case overhead, which captures all physical delays induced by the operating system during a single execution instance of task τ_i. It is decomposed into several distinct architectural penalties:

$$\Delta_i = \Delta_{sched} + \Delta_{cxs} + \Delta_{cpmd} + \Delta_{tick} + \Delta_{ipi} \tag{3}$$

Scheduling Overhead (Δ_{sched}): This represents the direct CPU time the RTOS kernel spends executing the scheduler routine. Under the EDF policy, the scheduler must dynamically calculate absolute deadlines and maintain a ready queue sorted by these deadlines. Depending on the RTOS implementation, inserting a task into an EDF queue can have a time complexity of $O(N)$ for simple linked lists or $O(\log N)$ for advanced red-black trees. In P-EDF, Δ_{sched} remains relatively small and predictable because the scheduler only manages a localized, short queue for a single core [5]. Conversely, in Global EDF (G-EDF), the scheduler must manage a massive, system-wide queue protected by heavy spinlocks to prevent concurrent modification by multiple cores, leading to severe lock contention and a drastically inflated Δ_{sched} [2], [5].

Context Switching Overhead (Δ_{cxs}): When the RTOS preempts a running task to execute a higher-priority one, it must perform a context switch. Δ_{cxs} encapsulates the latency associated with saving the hardware state (general-purpose registers, floating-point registers, program counters, and stack pointers) of the preempted task into its Process Control Block (PCB), and subsequently restoring the state of the incoming task [5]. Beyond the sheer register manipulation, context switching forces the processor to flush its execution pipeline and often triggers Translation Lookaside Buffer (TLB) misses, stalling the CPU while virtual-to-physical memory addresses are recalculated.

Cache-Related Preemption and Migration Delay (Δ_{cpmd}): Empirical studies consistently prove that memory and cache penalties are the single most dominant source of overhead in modern multi-core scheduling [2], [5]. When a task executes, it loads its instructions and data into the processor's fast L1 and L2 local caches (a "warm" cache state). If the task is preempted, the intervening higher-priority task will overwrite those cache lines with its own data. When the original task eventually resumes, it suffers a "cold" cache. The CPU must halt execution to fetch the evicted data all the way from the shared L3 cache or the main DRAM memory, which is orders of magnitude slower. This penalty is known as Cache-Related Preemption Delay (CRPD).

In Global EDF, this penalty is severely exacerbated into a Migration Delay. If a preempted task is pushed to a completely different core to resume execution, it completely loses its cache affinity [2]. It must drag its memory footprint across the system bus to the new core, heavily congesting the memory controller and inflating Δ_{cpmd}. Partitioned EDF (P-EDF) inherently neutralizes migration delays; because a task is statically bound to a specific core, its data frequently remains partially intact in the local L1/L2 caches across preemptions, keeping Δ_{cpmd} tightly bounded [5].

Timer Ticks (Δ_{tick}) and Inter-Processor Interrupts (Δ_{ipi}): Real-time kernels rely on periodic hardware timer interrupts (ticks) to track time, refill budgets, and monitor deadlines. Every tick forces the CPU to briefly jump to an Interrupt Service Routine (ISR), causing micro-delays (Δ_{tick}) that accumulate over a task's execution window [5]. Furthermore, multi-core systems rely on Inter-Processor Interrupts (IPIs) to allow one core to signal another. In G-EDF, if a task arrives on Core A but the lowest-priority running task is on Core B, Core A must send an IPI to force Core B to preempt. Handling these IPIs takes thousands of clock cycles (Δ_{ipi}). Because P-EDF features localized queues and autonomous core-level scheduling, it almost entirely eliminates the need for cross-core IPIs, vastly improving system determinism [5].

Integration into the Bin-Packing Phase: Since these

RTOS overheads are physically unavoidable, they must be integrated into the offline task partitioning phase. When algorithms like First-Fit Decreasing (FFD) attempt to assign tasks to cores, they cannot just use the theoretical utilization $u_i = C_i/T_i$. The allocation logic must calculate an inflated worst-case utilization for each task: $u'_i = (C_i + \Delta_i)/T_i$. By using u'_i during the FFD bin-packing phase, system designers can mathematically guarantee that the assigned workload will not crash the core's capacity even under the most pessimistic barrage of preemptions, cache evictions, and context switches [1], [5]. This rigorous integration of empirical overheads into mathematical bounds is what makes P-EDF the standard for safety-critical avionics and automotive platforms.

V. Performance: P-EDF vs. G-EDF

Empirical studies show that P-EDF achieves a better schedulability ratio in comparison to G-EDF in almost all possible distributions of tasks on actual multi-core systems [2], [5]. Poor performance of G-EDF is explained by the usage of one global lock for the ready queue along with migration flexibility, leading to higher costs of scheduling and CPMD processes. Analysis of the 99th percentile of the overhead per job in terms of CPU cycles shows significant differences in scalability [5]:

- **P-EDF Per-Job Δ:** 3,140 cycles on average, including 2,150 cycles of scheduling and 990 cycles of context-switching. This overhead demonstrates advantages of using local lock policies and maintaining cache affinity.
- **G-EDF Per-Job Δ:** up to 184,518 cycles on average, including 181,934 cycles of scheduling and 2,584 cycles of context-switching. This overhead is caused by high levels of global queue contentions and penalties of task migrations.

Thus, G-EDF could increase the WCET of a task almost 59 times compared to P-EDF. Partitioned scheduling outperforms global scheduling in hard real-time (HRT) domains due to lack of migration overheads such as cache invalidations and global lock contention delays [5].

VI. Conclusion

Though partitioned EDF algorithm is successful, this algorithm is far from perfect. It suffers from a few drawbacks, due to the nature of bin packing heuristics, which are not optimal, some processor cores might be heavily under-utilized, where other cores will reach their capacity [1]. Additionally, although the cost of task migrations in P-EDF is minimized to zero, the problem of resource sharing between cores (the L3 cache, memory buses, and interconnections) still remains, which may cause delays [2], [5]. The problem of achieving high utilization through the global schedule while keeping the overhead low is still being actively researched.

References

[1] S. Baruah, "Partitioned EDF scheduling: a closer look," *Real-Time Systems*, vol. 49, pp. 715–729, 2013.

[2] A. Bastoni, B. B. Brandenburg, and J. H. Anderson, "An Empirical Comparison of Global, Partitioned, and Clustered Multiprocessor EDF Schedulers," in *Proc. 2010 31st IEEE Real-Time Systems Symposium*, 2010.

[3] P. Han, W. Hu, Z. Zhai, and M. Huang, "A Model-Based Optimization Method of ARINC 653 Multicore Partition Scheduling," *Aerospace*, vol. 11, no. 11, p. 915, 2024.

[4] A. Burns, R. I. Davis, P. Wang, and F. Zhang, "Partitioned EDF scheduling for multiprocessors using a C=D task splitting scheme," *Real-Time Systems*, vol. 48, pp. 3–33, 2012.

[5] G. Gracioli, A. A. Fröhlich, R. Pellizzoni, and S. Fischmeister, "Implementation and evaluation of global and partitioned scheduling in a real-time OS," *Real-Time Systems*, vol. 49, pp. 669–714, 2013.